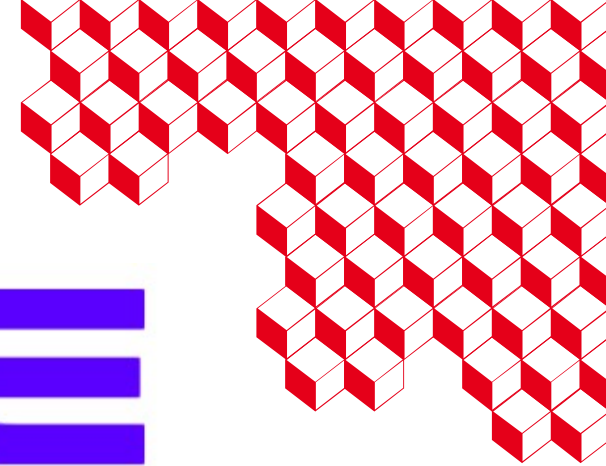




SALOME



Year 1 apprenticeship overview : SALOME Platform

Nicolas SAIKALY^{1,2}

1. Polytech Paris-Saclay

2. CEA - DES/ISAS/DM2S/SGLS/LESIM

université
PARIS-SACLAY



Supervised by : Christophe BOURCIER² , Mohd Afeef BADRI²

Wednesday, 17 June 2026

The CEA – brief presentation

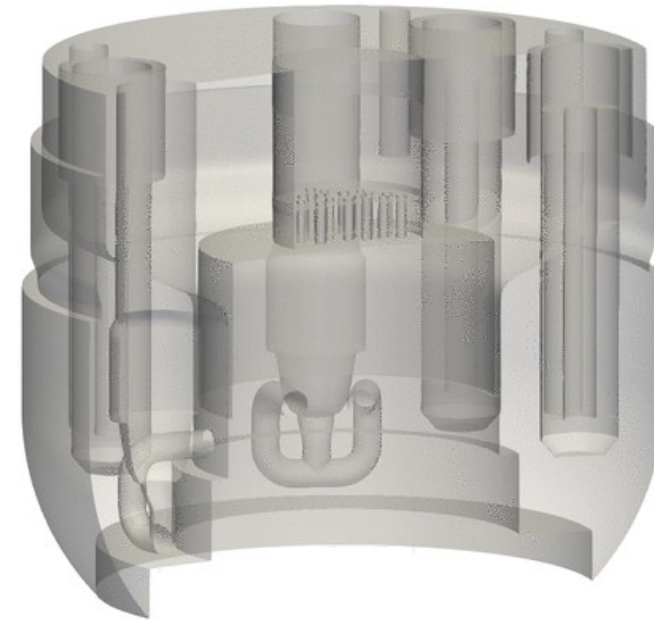
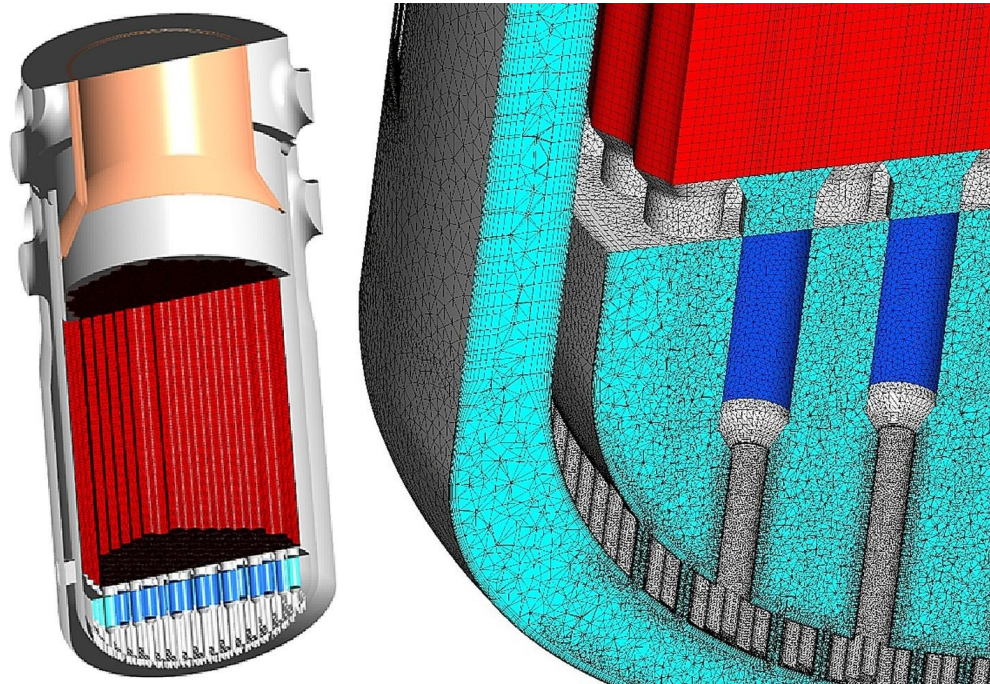
- **A major french public research organization**
 - Founded in 1945
 - Nuclear energy, defense & digital technologies
 - Multiple sites across France
- **CEA by numbers**
 - 21000 employees
 - Top 5 nationally for patents filed (598 in 2024)
 - Partner and founding member of Université Paris-Saclay
- **My host laboratory – SGLS/LESIM**
 - Software engineering and numerical simulations
 - Main open source project : SALOME simulation platform
 - Goal : provide a complete numerical simulation environment



Numerical simulations at CEA

- CEA heavily relies on numerical simulations for research
- Simulations replace costly, dangerous or impossible physical experiments
- Used in nuclear energy, fluid dynamics, thermal studies...

Example :



To make the pre-processing and post-processing of these simulations possible → CEA tool named SALOME

SALOME platform

■ Numerical simulation platform

- Co-developed since 2000
- Modular architecture
- Pre/Post-processing : geometry, meshing, visualization
- +5000 downloads per month – 2 major releases per year

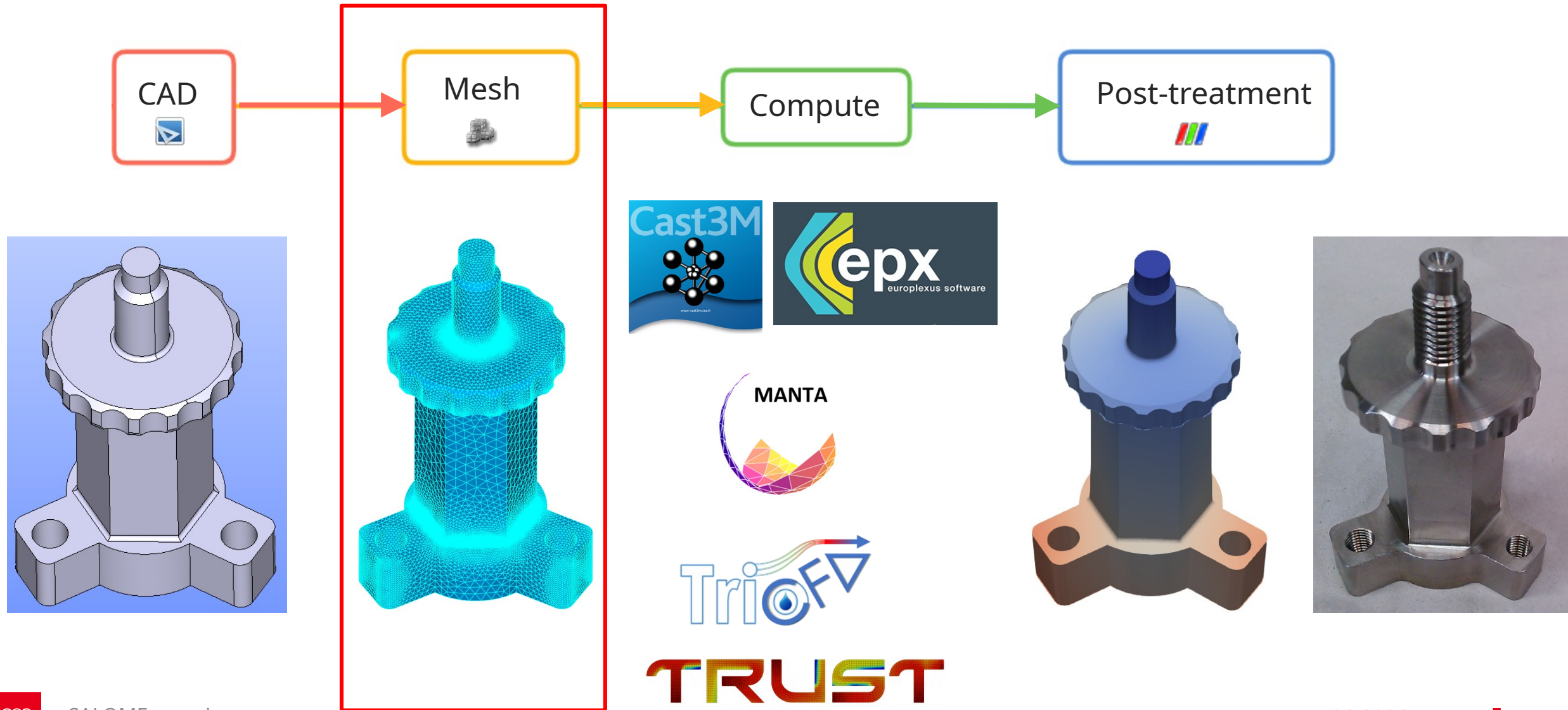


■ Academic and industrial use cases

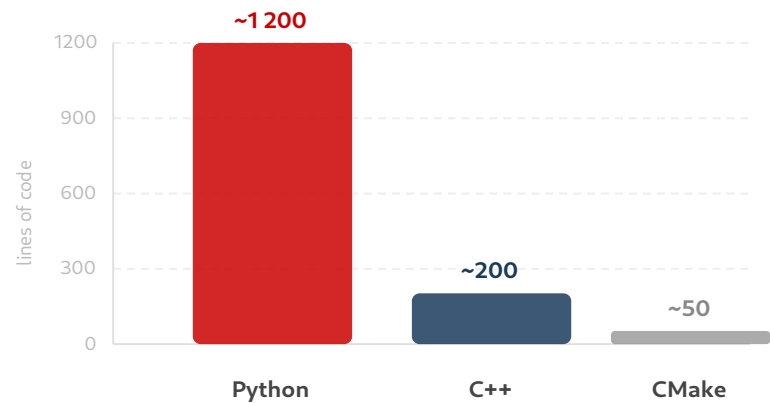
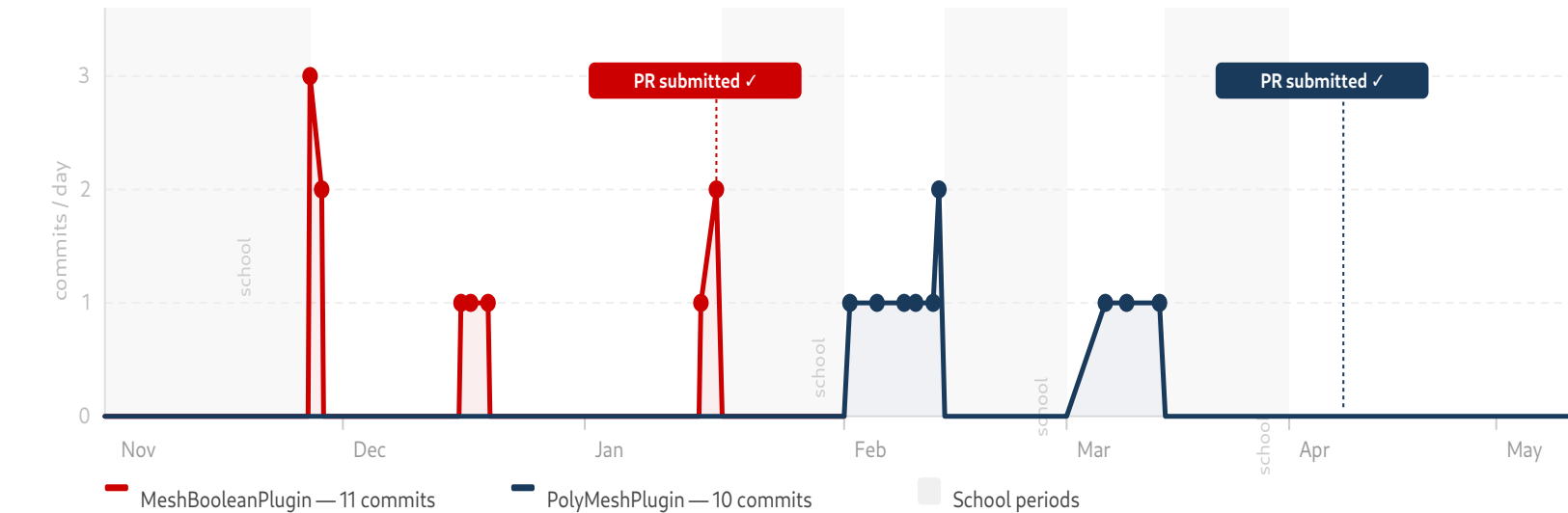
- Education & training in computational engineering
- Fundamental research in fluid dynamics & solid mechanics
- Energy, navy, aeronautics, automotive ...



Standard workflow



My contributions in numbers



- 21 commits
- 2 plugins developed
- 2 PR submitted
- ~1450 lines of code written
- 16 weeks of effective work

Agenda

1. Trimester 1 - MeshBooleanPlugin
2. Trimester 2 - PolyMeshPlugin
3. Conclusion & future work



1 ■ First trimester

MeshBooleanPlugin

■ Usage

- Performs boolean operations between 2 meshes : union, intersection, difference
- 6 C++ algorithms (CGAL, CORK, MCUT, libigl, IRMB, VTK)
- Outputs the resulting mesh directly in SALOME

■ Technical choices :

- Python – PyQt for the GUI – C++ algorithms
- Hosted on GitHub :
<https://github.com/SalomePlatform/meshbooleanplugin>
- Cross-platform : Linux & Windows

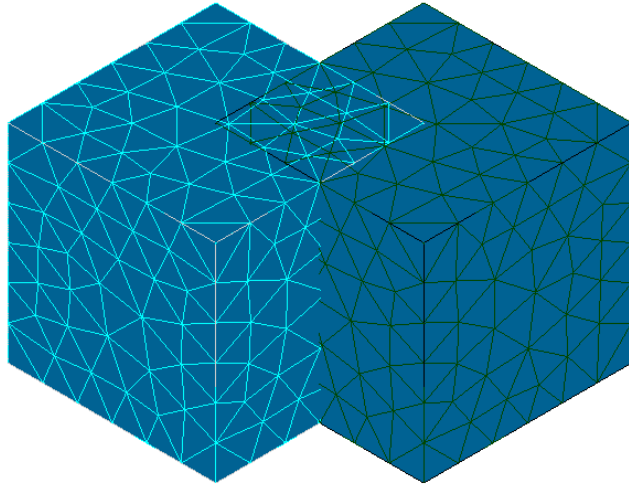
■ State when I took over :

- No way to cancel a running computation
- GUI and algorithm logic mixed together
- No API and dump study integration

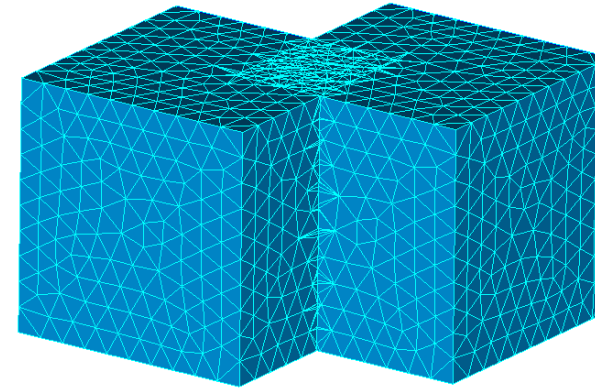


Types of boolean operations

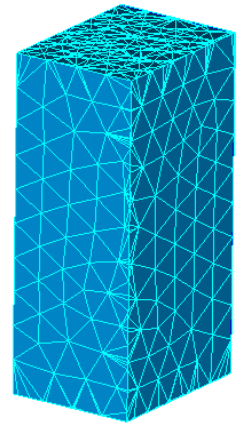
■ Input meshes :



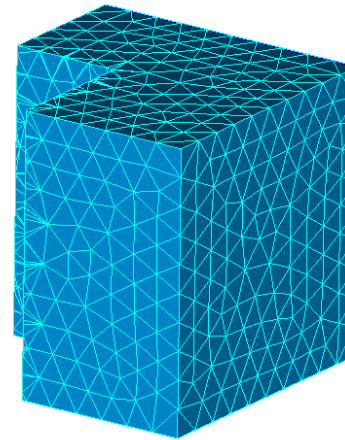
■ Union
combined volume



■ Intersection
overlapping region only



■ Difference
A minus B



Cancel button – First technical challenge

The problem

Computation started → no way to stop it – wait or kill SALOME entirely

Solution

- First attempt – multiprocessing
 - Worked on linux, failed on windows
 - **First lesson : SALOME must be fully cross-platform**

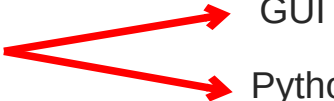
Search for cross-platform method → standard python library

- Second attempt – subprocess + QThread
 - Subprocess : launches the algorithm as an external process
 - Qthread : monitors the process asynchronously
 - Qt signal sent to the worker thread on cancel click
 - **Same behavior on windows and linux**

Add Cancel Button #22

 Merged [c-bourcier](#) merged 3 commits into [SalomePlatform:main](#) from [Nicolas-Saikaly:nsy/add_cancel](#)  on Nov 27, 2025

Python API & GUI independance

SALOME can be used via 

GUI
Python scripts

The API allows using the plugin in those scripts

■ Decoupling logic & UI :

- Extraction of algorithms into a standalone python API
- PyQt interface a lightweight wrapper calling the API

■ CLI & tests :

- Full command line execution without lanching SALOME
- Algorithm precision validated at 5×10^{-4} margin

■ Code quality and robustness

- Pylint for code cleanup
- Temporary file cleanup via context managers – guaranteed even on crashes



```
# We now take care of everything related to temporary files management in this fonction rather than in the GUI
try:
    with tmpDir() as tmp_path:
        # the with assures that the directory is deleted after the compute or if there is an exception
        print(f"Temporary directory created: {tmp_path}")
        # Convert left and right

        objL = exportToObj(mesh_left, tmp_path)
        objR = exportToObj(mesh_right, tmp_path)

        med_result = tmpFile(".med", tmp_path=tmp_path)

        # call runAlgo
        process = runAlgo(algo,
                          operator_name.lower(),
                          objL,
                          objR,
                          med_result
                          )

        if worker is not None:
            worker.process = process
        # Wait the end of the process if there is one
        if process :
            rc = process.wait()
            if rc != 0:
                if worker and not worker.isRunning:
                    print("Process killed by user")
                    return None
                raise RuntimeError("Boolean operation ended in error")

        if worker and not worker.isRunning:
            return None

        #Convert the result
        convertAlgorithmResult(algo, med_result)

        #Import in SALOME
        result_mesh = importMedToSmesh(med_result, operator_name = operator_name, name = name)
```

Dump study integration

- Records all GUI actions of the user as a Python script → Allows user to replay operations without manual interaction
- Intermediate file exports polluted the generated script → dump unusable out of the box

■ My contribution

- Pause recording during intermediate export steps
- Inject clean API calls directly into the dump
- GUI actions recorded as proper API commands

■ Result

- Replay or automate boolean operations from the script
- Bridges the gap between GUI and python API

```
from meshbooleanplugin import mesh_boolean_api
union_1 = mesh_boolean_api.Union(Mesh_1.GetMesh(), Mesh_tetra_coarse.GetMesh(), algo = mesh_boolean_api.CGAL)
intersection_1 = mesh_boolean_api.Intersection(Mesh_1.GetMesh(), Mesh_tetra_coarse.GetMesh(), algo = mesh_boolean_api.VTK)
difference_1 = mesh_boolean_api.Difference(intersection_1, Mesh_tetra_coarse.GetMesh(), algo = mesh_boolean_api.CORK)
```

Scientific use cases

■ Why boolean operations on meshes?

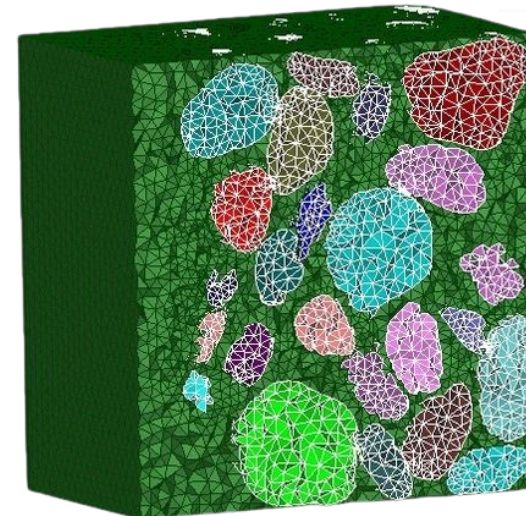
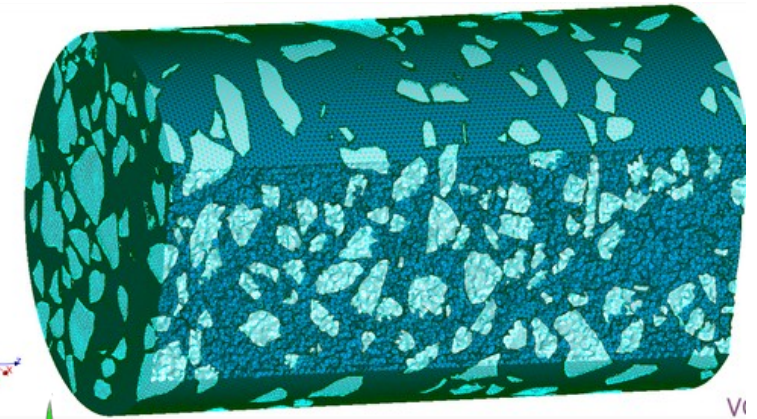
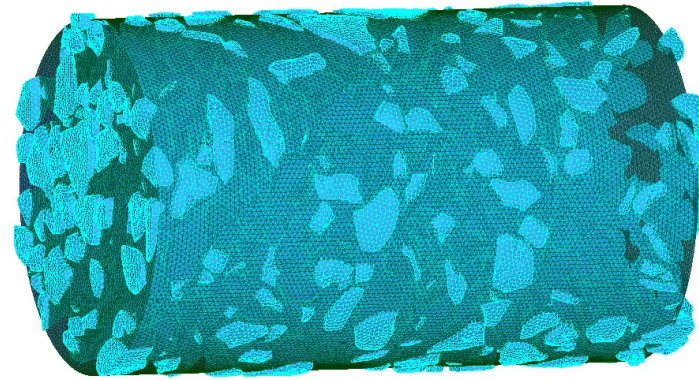
- Essential step in numerical simulation workflows
- When no CAD model is available – e.g tomography data

■ Concrete & aggregates

- Intersection of a cylindrical surface mesh with concrete aggregates
- Used to model mechanical behavior of composite materials
- Mesh clipped to visualize interior volume

■ Lithium-ion battery cells

- Boolean operations between cell components
- Can be used in battery research simulations at CEA





2 ■ Second trimester

PolyMeshPlugin - overview

■ Usage

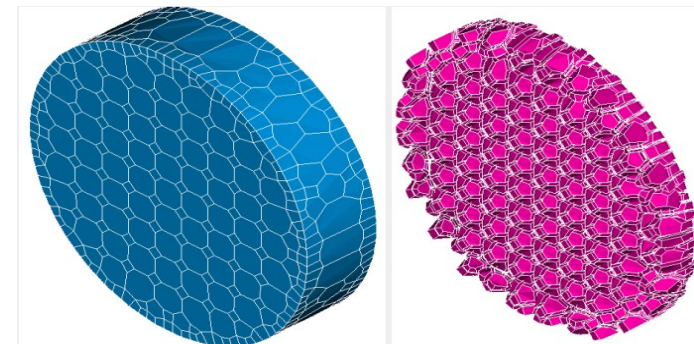
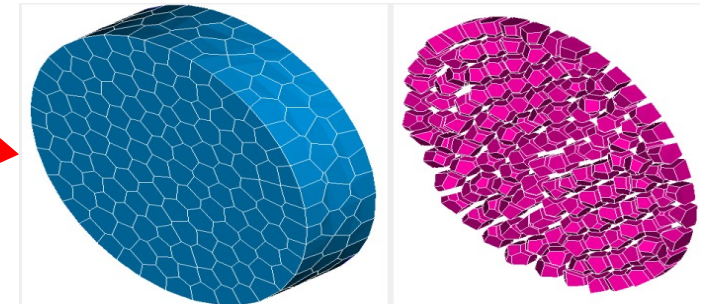
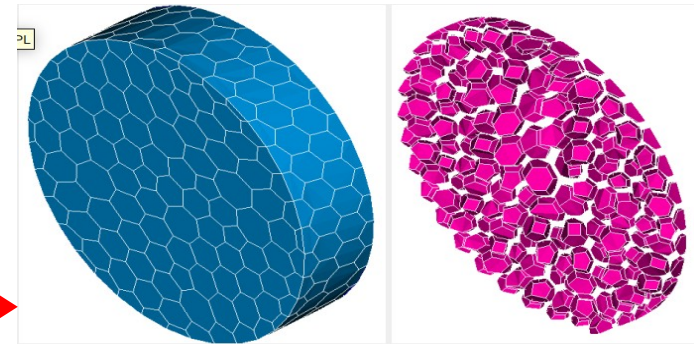
- Transforms any mesh into a polyhedral mesh
- 3 integrated C++ algorithms
 - Polydual
 - Geogram
 - Cfmesh

■ Technical choices :

- Python – PyQt wrapper – C++ algorithms
- Cross-platform : Linux & Windows

■ State when I took over the plugin

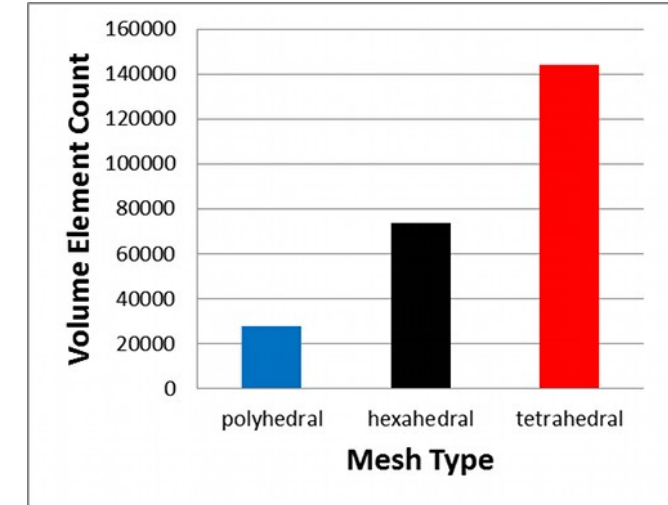
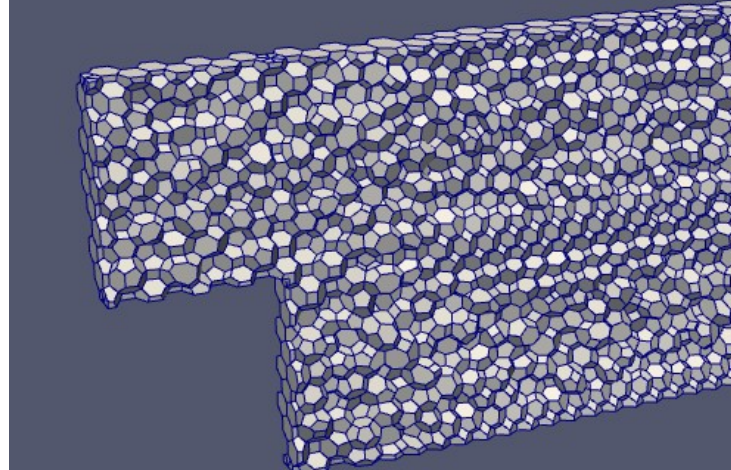
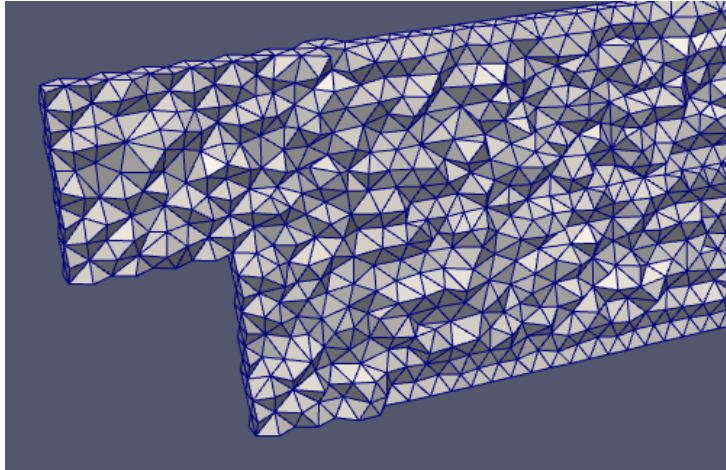
- Complications on windows
- GUI freezing after computation
- No proper error handling



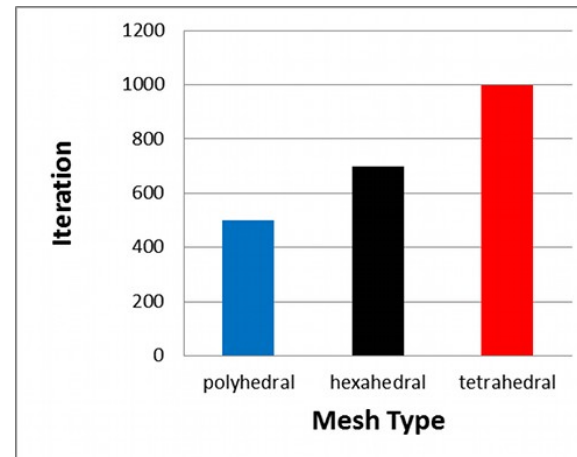
Advantage of polyhedral meshing

- Lower cell counts → faster computation

Statistics from [sym scape](#)



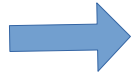
- Better orthogonality
- Better fit on complex geometries
- The sweet spot between :
 - Tetra (easy to generate/worst cfd result)
 - Hexa (hard to generate/better cfd result)



Cross-platform refactoring



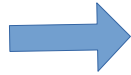
wexpect/pexpect – requires a post
install script on windows



Subprocess – standard python
library

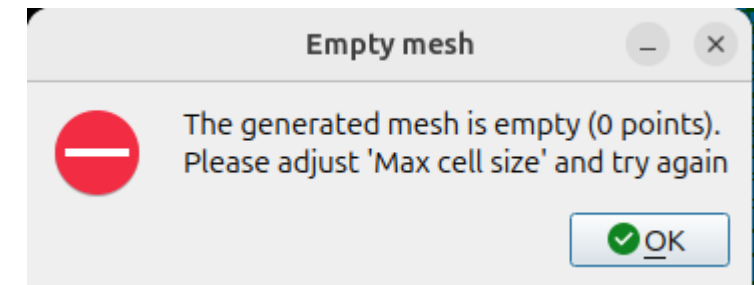
■ Same behavior on Linux and Windows

Real time progress bar



■ Character by character reading of process output

- Loading message box after computation ends – user always knows what is happening
- Explicit check on number of generated mesh points
- Partial group transfer triggers a warning instead of a silent failure



Geogram – architectural overhaul

BEFORE

Geogram → .ovm → python parsing → SALOME

Reason for the change

- Delete hundreds of lines of python parser
- Python layer will simply import .med natively
- Faster, stable and maintainable

From .ovm to .geogram

- Developed a dedicated C++ executable - geogram2med
 - Reads .geogram file by extracting nodes and groups facet_id
 - Reconstructs polyhedra via MEDCoupling
 - Exports directly to .med

AFTER

Geogram → .geogram → geogram2med → .med → SALOME



3 ■ Conclusion & future work

Takeaways

■ Personal growth

- Linux autonomy
- Modern scientific C++ training – CNRS, 21 hours
- Immersion in a scientific environment

■ Technical skills

- GUI development – PyQt, widgets, signals, Qthread
- Software architecture – API design
- Python/C++ integration
- Code quality – tests , pylint, git rebase

■ Pull requests status

- Successfully merged into each plugin's master
- Available in next SALOME release

Perspectives and future tasks

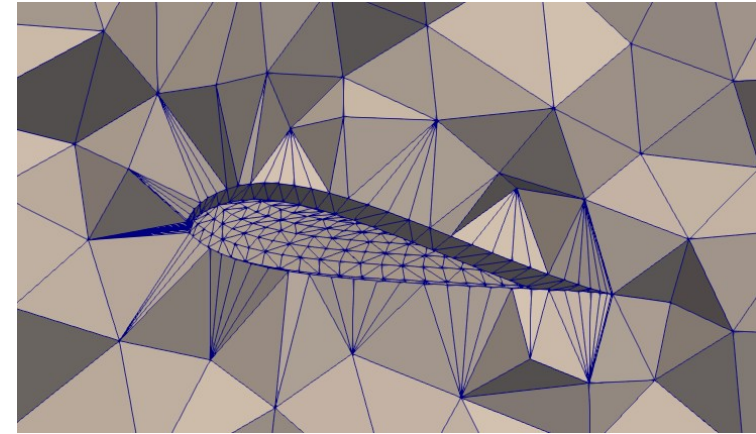


■ Overall objective

- Build expertise progressively over 3 years
- Take on increasingly complex engineering problems
- Contribute to SALOME as a production ready platform

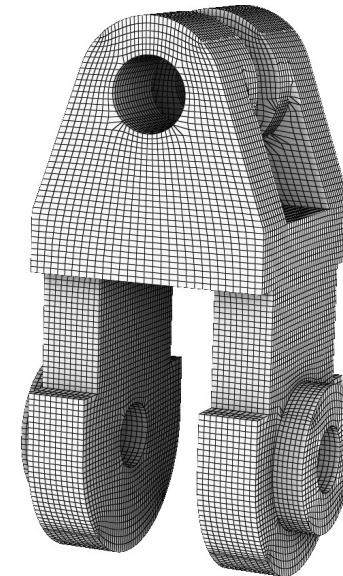
■ Code_saturne

- Already started exploring code_saturne's prismatic layer algorithm
- Goal : integrate it as a new SALOME plugin alongside PolyMeshPlugin
- Generate prismatic layers directly from the GUI



■ Hexahedral meshing

- Multi-tool workflow integration in SMESH
- Based on PhD work from LESIM/LIHPC





Thank you !

Questions ?



CEA SACLAY

91191 Gif-sur-Yvette Cedex

France

nicolas.SAIKALY@cea.fr